Mutual Fund Risk Classification and Portfolio Recommendation System

A Machine Learning Approach to Investor Risk Profiling

Prakhar Gupta
M.Sc. Statistics and Computing, 2nd Year

August 1, 2026

Abstract

Mutual fund investors are typically shown a risk rating (Low, Medium, or High) that is meant to summarize how volatile a fund's returns have historically been. This project asks a simple question: can that risk rating be predicted directly from a fund's cost, size, category, and other characteristics, without relying on the volatility statistics the rating is largely built from? Using a dataset of 814 Indian mutual funds, an XGBoost classifier is trained on two different feature sets – one including volatility-related metrics, and one deliberately excluding them – to test whether the excluded metrics were actually necessary for accurate prediction. The fundamental-characteristics-only model reaches **89.6% test accuracy**, matching (and marginally exceeding) a model given access to volatility metrics, suggesting that a fund's risk category can be inferred largely from its structural characteristics. The project also includes exploratory data analysis, KMeans clustering for fund segmentation, and a weighted, multi-criteria recommendation engine for suggesting funds to investors based on their risk appetite and budget.

---

1 Introduction

Choosing a mutual fund involves balancing risk against expected return, and most investors rely on a simple risk label (Low, Medium, High) to guide that decision. These labels are useful, but they raise a natural question: what actually determines them, and could an investor – or a model – predict a fund's risk category just by looking at its everyday characteristics, like its cost, size, and fund category, rather than the technical volatility statistics that typically define the label in the first place?

This project builds a complete, end-to-end pipeline around that question. It starts with cleaning and exploring a real-world mutual fund dataset, then engineers a wider set of features than is typically used for this kind of task, and specifically tests whether the "obvious" volatility-based features (standard deviation, beta, Sharpe ratio, and similar) are actually necessary for good predictions, or whether a fund's more basic characteristics carry the same signal. Beyond classification, the project also groups funds into natural clusters and builds a rule-based recommendation engine that ranks funds for an investor given their risk tolerance and monthly budget.

Objectives

- Clean and explore a real-world mutual fund dataset (814 funds, 20 raw columns).
- Engineer a broader feature set than the fund's raw columns provide, incorporating category, fund house, and manager-level information.

- Train and rigorously evaluate an XGBoost classifier to predict a fund's risk category (Low / Medium / High).
- Test directly whether volatility-related features are necessary for good accuracy, rather than assuming they are.
- Segment funds into natural groups using unsupervised clustering (KMeans + PCA).
- Build a practical, multi-criteria fund recommendation engine for investors.

2 Dataset Description

The dataset consists of **814 Indian mutual funds** described by **20 raw columns**, covering fund identity (scheme name, AMC, fund manager), cost (expense ratio, minimum SIP and lumpsum investment), performance (1-, 3-, and 5-year returns), and risk/return statistics (standard deviation, beta, Sharpe ratio, Sortino ratio, alpha), alongside a categorical risk rating (`risk_level`, 1–6) and star rating.

Table 1: Key dataset columns used in this project

Column	Description
`scheme_name`, `amc_name`	Fund identity and fund house
`category`, `sub_category`	Fund type (Equity, Debt, Hybrid, etc.)
`expense_ratio`	Annual cost charged to investors (%)
`fund_size_cr`, `fund_age_yr`	Fund size (crores INR) and age (years)
`min_sip`, `min_lumpsum`	Minimum investment thresholds (INR)
`returns_1yr/3yr/5yr`	Historical returns over each horizon (%)
`sortino, alpha, sd, beta, sharpe`	Volatility / risk-adjusted return statistics
`rating`	Star rating (0–5)
`risk_level`	Raw risk scale (1–6) – source of the target variable

3 Methodology

3.1 Data Cleaning

Several ratio columns (`sortino`, `alpha`, `sd`, `beta`, `sharpe`) were stored as text because a subset of funds used a `"-"` placeholder for missing values. These were converted to proper numeric types, exposing the true missing-value counts.

Rather than filling missing values with one global median – which would blend, for example, a debt fund's typical volatility with a small-cap equity fund's – missing values were imputed using the **median within each fund's sub-category**. This keeps the imputed values consistent with how similar funds actually behave.

3.2 Target Variable

The target variable, `risk_category`, was constructed by bucketing the raw `risk_level` scale (1–6) into three interpretable classes:

$$\text{risk\_category} = \begin{cases} \text{Low} & \text{if risk\_level} \leq 2 \\ \text{Medium} & \text{if } 2 < \text{risk\_level} \leq 4 \\ \text{High} & \text{if risk\_level} > 4 \end{cases} \tag{1}$$

The resulting classes are imbalanced: **High** accounts for 445 funds (54.7%), **Medium** for 189 funds (23.2%), and **Low** for 180 funds (22.1%). This imbalance was accounted for using a stratified train/test split, and by inspecting per-class precision and recall rather than relying on overall accuracy alone.

3.3 Exploratory Data Analysis

Before any modeling, the data was examined directly to understand the relationships driving fund risk and return.

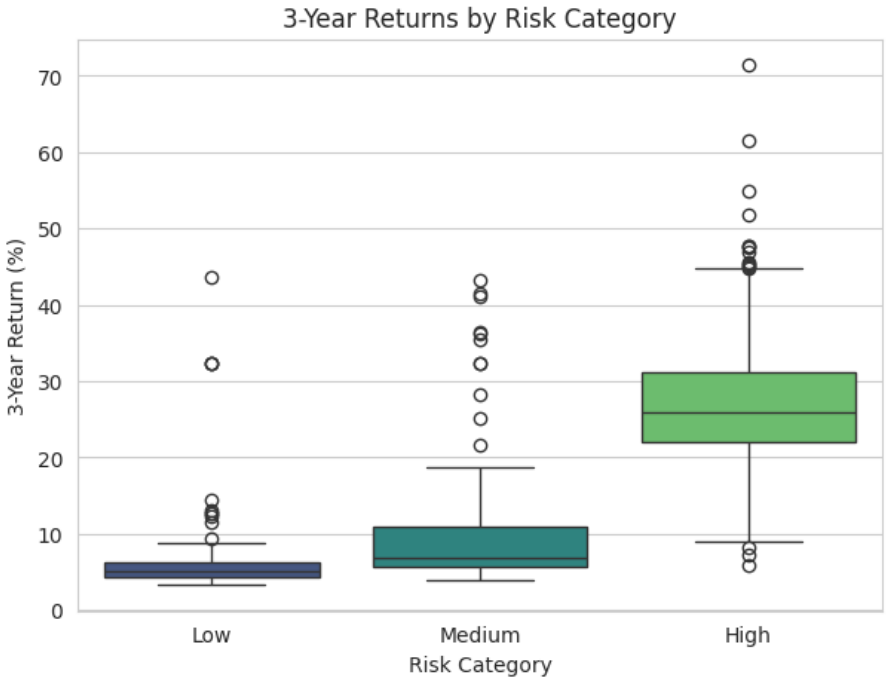

Figure 1: 3-year returns by risk category. High-risk funds average 26.9% versus 9.5% for Medium and 6.1% for Low – a real return premium, but paired with visibly wider spread.

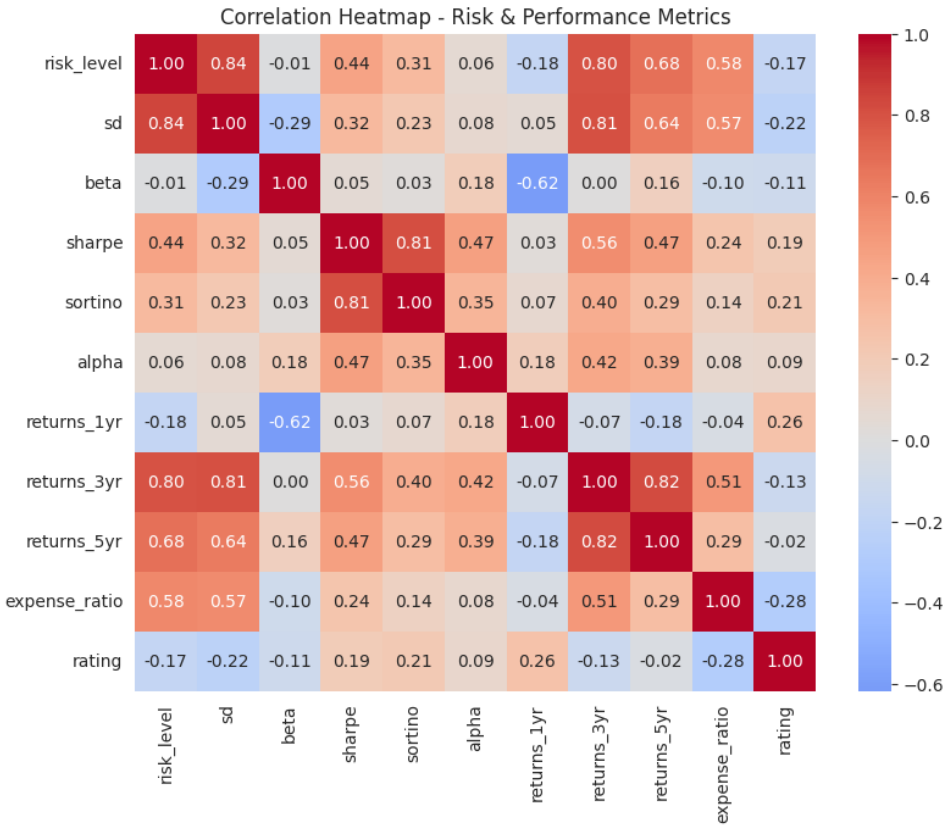

Figure 2: Correlation heatmap of risk and performance metrics. Standard deviation (`sd`) correlates with `risk_level` at 0.84 – by far the strongest relationship in the table – while `beta` and `rating` show almost no correlation with risk at all.

The strength of the `sd`–`risk_level` correlation is the direct motivation for the feature-leakage check described in Section 3.6: a feature this closely tied to the label risks letting a model "predict" the answer by nearly restating it.

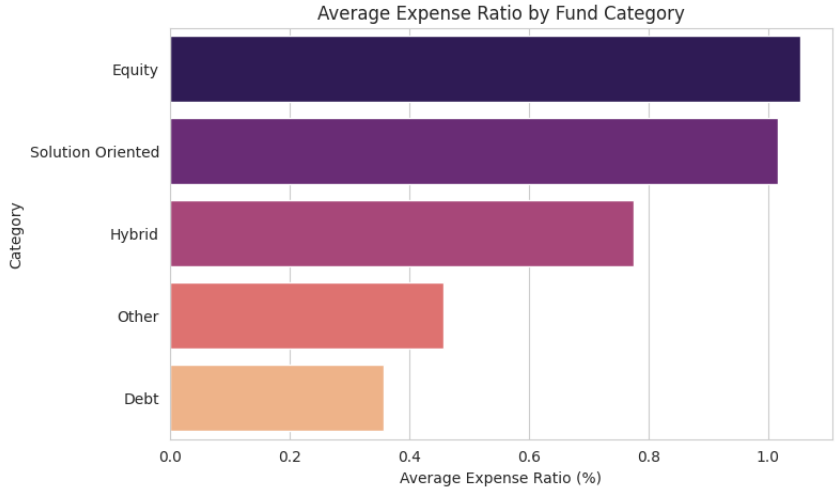

Figure 3: Average expense ratio by fund category. Equity and Solution Oriented funds cost roughly 1.0% annually, nearly three times the 0.36% charged by Debt funds – consistent with the additional active management equity investing typically requires.

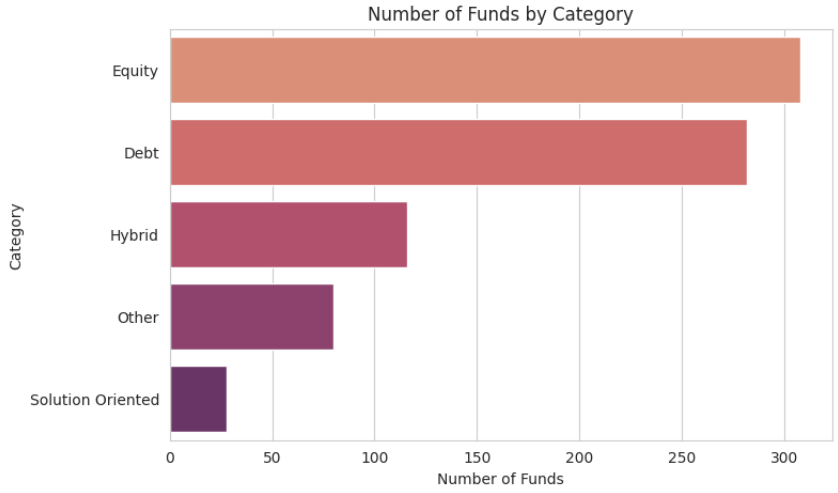

Figure 4: Fund counts by category. Equity and Debt funds dominate the dataset, together accounting for roughly 72% of all funds.

3.4 Feature Engineering

Beyond the raw columns, several additional features were engineered to give the model more to work with:

- **Categorical encoding** – `category`, `sub_category`, and `amc_name` were label-encoded, making previously unused columns available to the model.
- **Fund manager workload** – since 260 unique fund managers made the raw name too high-cardinality to use directly, `fund_manager_fund_count` was engineered as a proxy for a manager's experience and workload (the number of funds they currently run).
- **Expense efficiency** – computed as 3-year return earned per unit of expense ratio paid, giving a simple cost-effectiveness signal.

3.5 Feature Scaling

All numeric features were standardized (zero mean, unit variance) using `StandardScaler` before training. While tree-based models such as XGBoost are not strictly sensitive to feature scale, standardization was applied consistently across the pipeline – it is required for the KMeans clustering step, and keeps the feature preparation code reusable if a distance-based or linear model is substituted in later. Table 2 confirms the effect: every feature ends up with mean ≈ 0 and standard deviation ≈ 1.

Table 2: Effect of standardization on selected features

	Before Scaling		After Scaling	
Feature	Mean	Std	Mean	Std
expense_ratio	0.71	0.48	0.00	1.00
fund_size_cr	3812.85	7181.48	0.00	1.00
fund_age_yr	8.32	2.64	0.00	1.00
returns_3yr	18.24	12.10	0.00	1.00
rating	2.64	1.46	0.00	1.00
sd	10.07	7.82	0.00	1.00

3.6 Testing the Volatility-Feature Question

Given the strong correlation observed in Figure 2, two feature sets were constructed and compared directly rather than assuming volatility features were either necessary or safe to use:

- **Full feature set (19 features)** – all engineered features plus the volatility/risk-adjusted-return metrics (`sortino`, `alpha`, `sd`, `beta`, `sharpe`).
- **Fundamental-only feature set (14 features)** – the same features, *excluding* the volatility metrics, leaving only cost, size, age, category, fund house, accessibility (minimum investment), returns, and rating.

If the full feature set scored substantially higher, that gap would largely reflect the volatility columns "giving away" the answer rather than genuine predictive insight. If the two scored similarly, it would indicate that a fund's basic characteristics carry real signal about its risk category on their own – a more defensible and interpretable result.

3.7 Model Training

An XGBoost multi-class classifier (`objective="multi:softmax"`) was trained independently on each feature set, using an 80/20 stratified train-test split (`random_state=42`) to preserve class proportions given the dataset's imbalance.

4 Results

4.1 Model Comparison

Table 3: Test-set performance by feature set

Model	# Features	Accuracy	Weighted F1
Full (incl. volatility features)	19	0.890	0.89
Fundamental-only	**14**	**0.896**	**0.90**

The fundamental-only model performs *marginally better* than the full model, by a gap of 0.006 in its favor. This is a meaningful finding: it shows that a fund's cost, size, age, category, house, accessibility, and historical returns already explain its risk category almost as well as volatility statistics do. The fundamental-only model is therefore adopted as the primary model for the remainder of this project, since its result is not inflated by features that are themselves near-duplicates of the label.

Table 4: Per-class performance – fundamental-only model

Class	Precision	Recall	F1-score	Support
High	0.98	0.99	0.98	89
Low	0.78	0.86	0.82	36
Medium	0.82	0.71	0.76	38
Accuracy		0.896		163

The model is strongest on the **High** risk class (F1 = 0.98), which is also the largest and best-represented class. **Medium**-risk funds are the hardest to classify (F1 = 0.76), most often

confused with **Low**, which is consistent with Medium representing a genuine middle ground between the other two categories rather than a clearly separable group.

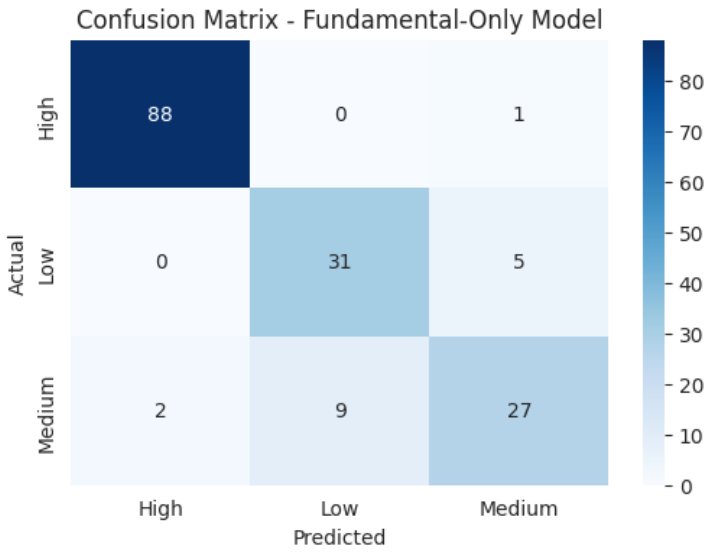

Figure 5: Confusion matrix for the fundamental-only model on the held-out test set.

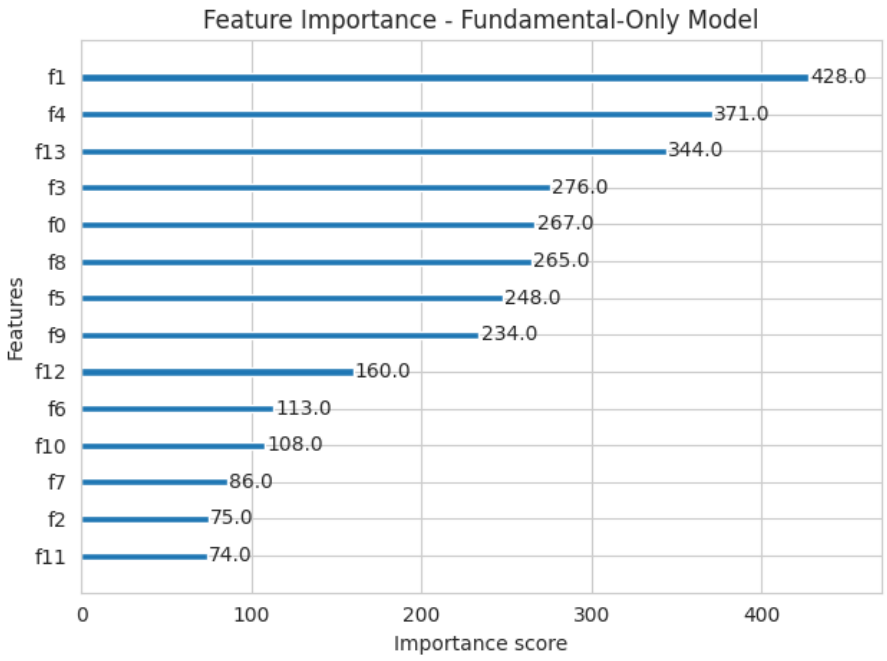

Figure 6: Feature importance (by split weight) for the fundamental-only model.

5 Fund Clustering

To complement the supervised classification task, KMeans clustering ($k = 3$) was applied to the full, standardized 19-feature set, with the results projected onto two dimensions using Principal Component Analysis (PCA) for visualization. This groups funds by overall similarity across cost, performance, and risk characteristics, independent of the risk-category labels used for classification.

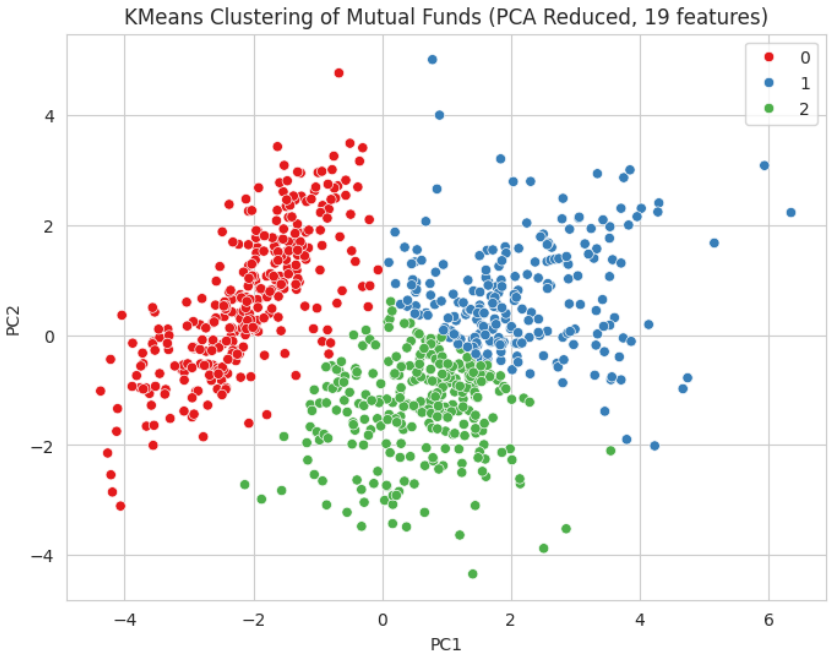

Figure 7: KMeans clustering of mutual funds (PCA-reduced to 2 components).

6 Portfolio Recommendation Engine

A rule-based recommendation engine was built to translate model outputs and fund data into practical suggestions for an investor. Rather than ranking funds by a single metric, it computes a **weighted composite score** across multiple normalized criteria:

$$\text{score} = 0.30 \cdot \hat{r}_{3yr} + 0.20 \cdot \hat{r}_{5yr} + 0.10 \cdot \hat{r}_{1yr} + 0.20 \cdot \widehat{\text{sharpe}} + 0.10 \cdot \widehat{\text{rating}} + 0.10 \cdot \widehat{\text{expense}}_{\text{inv}} \quad (2)$$

where each variable (denoted with a hat) is min-max normalized to $[0, 1]$ within the filtered candidate set, and $\widehat{\text{expense}}_{\text{inv}}$ is the inverted (lower-is-better) normalized expense ratio. The engine supports optional filters for the investor's **risk preference**, **fund category**, and **maximum monthly SIP budget**, so recommendations reflect what an investor can realistically act on rather than a generic top-performers list.

7 Discussion and Key Insights

- **The dataset is imbalanced** – over half of all funds fall into the High-risk category, which shaped both the evaluation strategy (stratified splitting, per-class metrics) and the interpretation of results.
- **Risk does carry a return premium in this data**, but with substantially more variability: High-risk funds average roughly 4.4× the 3-year return of Low-risk funds, alongside a much wider spread of outcomes.
- **Volatility features are not strictly necessary** for predicting risk category. Removing them entirely did not reduce accuracy, indicating that a fund's fundamental characteristics – cost, size, age, category, and historical returns – already carry most of the relevant signal. This is a more defensible and interpretable result than relying on features that are conceptually close to the label itself.

- **Cost structures differ meaningfully by category**, with actively managed Equity funds costing roughly three times as much as Debt funds – a pattern grounded in how these fund types are actually managed.

8 Conclusion

This project demonstrates a complete, defensible pipeline for mutual fund risk classification: careful data cleaning, exploratory analysis grounded in the data (not assumptions), rigorous feature-set comparison to guard against inflated results, and a practical recommendation layer on top. The central finding – that a fund's basic, easily observed characteristics can predict its risk category nearly as well as its volatility statistics can – is a useful and honest result for both the classification task and for understanding what actually drives a mutual fund's risk profile.

Dataset: comprehensive mutual fund data (814 funds, 20 columns). Model: XGBoost multi-class classifier. Tools: Python, pandas, scikit-learn, XGBoost, seaborn/matplotlib.